

A Distributed Sorting Framework for Ad Hoc Wireless Sensor Networks

Amitabha Ghosh

Honeywell Technology Solutions Lab
151/1 Doraisanipalya, Bannerghatta Road
Bangalore 560076, India
Email: amitabha.ghosh@honeywell.com

Balamurugan Mohan

Honeywell Technology Solutions Lab
TCE Campus, Thiruparankundram
Madurai 625015, India
Email: balamurugan.mohan@honeywell.com

Abstract—Wireless sensor networks (WSN) are distributed, self-organizing, pervasive systems, which perform collaborative computations to provide useful information about the underlying stochastic phenomenon. In this paper, we explore the ranking and sorting problems in distributed sensor networks which provide perspectives in understanding certain fundamental issues in WSN, as well as motivate many exciting real life applications. In the first part, we propose a simple, distributed and parallel sorting algorithm in single-channel, multihop, lattice sensor networks that has a time complexity of $O(\sqrt{n})$ and an energy complexity of $O(n\sqrt{n})$. In the sequel, we consider the sorting problem under a generalized framework of geometric random graphs and show that finding a time and/or energy optimal distributed sorting algorithm is NP-hard. Finally, we prove the existence of a randomized sorting algorithm under the generic setup.

I. INTRODUCTION

Sorting is a fundamental problem in computer science that has been analyzed in great details over the years. We see the value of addressing this problem in distributed sensor networks from two perspectives. Firstly, distributed sorting being a collaborative process, analyzing it for sensor networks will allow us to understand certain fundamental issues. When nodes collaborate to solve certain problems in a distributed manner, under settings like sorting, we will be able to understand many design issues and limitations, and also find approaches to solve them efficiently. At the same time, we will be able to explore the interaction between formation of networks, which we model as a graph, synchrony and asynchrony in media access protocol, broadcast nature of communication, routing and flow of data. The second perspective to the problem comes from certain application requirements. Since in wireless sensor networks, nodes collect data samples from the environment, many of which might be erroneous, and hence, they are often needed to be sorted in order to filter out extreme values. Some of the applications require ranking of sensor nodes based on specific metrics. For example, the nodes might be ranked according to their residual battery power, so that accordingly they can adjust their transmission ranges to optimize power consumption. In some other application scenarios, it is not only necessary for each sensor node to know its own rank, but also to have the knowledge which of its neighboring nodes

are ranked higher or lower than itself. Another example is target tracking with sensor networks, where many sensors near the target will get activated in the presence of a target and all of them will perform the detection. However, since the data or the measurement in those nodes will be spatio-temporally correlated, it would make sense to quantitatively evaluate such measurements and assign a value according to the merit of the data. Such a merit could be based on information theoretic measures, like mutual information content between sensor values. Once the measurements are assigned values, the aggragation mechanism may just need to rank and sort the values so that they could be aggregated meeting the objective.

To our knowledge, the sorting problem in sensor networks or similar broadcast networks, like packet radio networks (PRN), so far has been considered under restricted scenarios. In [1], Bordim et. al proposed a sorting algorithm, which uses basic operations of comparing either in rows or columns and a variant of merge sort to sort n data elements placed in n sensor nodes arranged in a two-dimensional grid of size $\sqrt{n} \times \sqrt{n}$ in $O(r\sqrt{n})$ time slots, for $r < \sqrt{n}$, where r is the transmission radius. For such short transmission ranges, this time complexity matches with that of the optimal algorithm proposed by Nassimi and Sahni in [2], which is an adaptation of bitonic sort. The authors in [2] had shown that on a mesh connected parallel computer, n^2 elements can be sorted in $O(n)$ time. However, for large transmission ranges ($r \approx \sqrt{n}$) since the number of nodes that can transmit simultaneously decreases to avoid interference, Bordim's algorithm can take as much as $O(r^2)$ time slots.

Singh et. al in [3] emulated some of the optimal and distributed parallel algorithms in two-dimensional grid sensor networks and showed that an energy balanced ranking algorithm would take $O(r^2 + \sqrt{n}/r)$ time with overall energy dissipation $O(nr^2)$. They also demonstrated that using a clustering approach bitonic sort can be implemented on a two dimensional grid of size $\sqrt{n} \times \sqrt{n}$ with time complexity $O(r^2 + r\sqrt{n})$ and with energy complexity $O(nr^2 + rn\sqrt{n})$, which conform to the results as obtained in [1]. The same authors in [3] presented an energy-optimal algorithm for sorting in a single-hop single-channel sensor network and showed that their algorithm can sort n randomly distributed

sensors in $O(n \log n)$ time and energy, with no sensor being awake for more than $O(\log n)$ time steps.

However, all the afore-mentioned works have considered the ranking and sorting problems under very restricted scenarios, either by assuming a single-hop sensor network or by imposing a grid structure on a sensor network. It should be noted that a single-hop WSN is of little practical importance because in most of the real world deployments, there will be multi-hop WSN to be considered. Similarly, grid structures imposed on sensor networks restrict the feasibility of the algorithms in practical applications, because of the fact that most sensor networks are formed by ad hoc random deployment of nodes rather than following any geometric structure. In this paper, we consider the sorting problem in a more generic setup and assume comparison-only algorithms. We also observe that the efficiency of the ranking and sorting algorithms is dependent on the transmission radii of the sensor nodes. A large transmission radius reduces the network diameter but at the same time, results in higher energy dissipation and reduces the number of nodes that can transmit or receive concurrently because of interference.

The rest of the paper is organized as follows. In section II we formally introduce the model and the problem statement for describing a simple, distributed and parallel sorting algorithm under the restricted scenario of single-channel grid sensor network. Next, in section III we formulate the sorting/ranking problem under the generic setup of random deployment and show that finding a time and/or energy optimal distributed sorting algorithm is NP-hard. We also prove the existence of a schedule of exponential length that allows all the sensor nodes to transmit atleast once with very high probability.

II. RESTRICTED: GRID-BASED, SINGLE-CHANNEL WSN

A. Problem statement

In this section, we consider the sorting problem on a WSN under a restricted scenario, where nodes are arranged in the form of a two-dimensional grid and there exists a single channel for communication. Let n be the total number of nodes placed on a grid of size $r \times c$, where r is the number of rows and c is the number of columns. Each node has a data element and a unique identifier stored in it. Let us also associate a virtual identifier ij , with each node, depending on its position in the grid. Identifier ij uniquely determines a node placed on row i and column j , where $1 \leq i \leq r$ and $1 \leq j \leq c$. Let us denote the value stored in node ij by v_{ij} . The goal of the sorting algorithm is to redistribute the data among the nodes, such that at the end of the algorithm the following criteria holds.

1) when i is odd:

- $v_{i(j+1)} \geq v_{ij}$, for $1 \leq j < c$
- $v_{(i+1)c} \geq v_{ic}$, for $1 \leq i < r$
- $v_{(i-1)1} \leq v_{i1}$, for $1 < i \leq r$

2) when i is even:

- $v_{i(j+1)} \leq v_{ij}$, for $1 \leq j < c$
- $v_{(i+1)1} \geq v_{i1}$, for $1 < i < r$

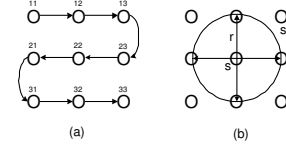


Fig. 1. (a) Row-major snake-like pattern, (b) Communication radius

- $v_{(i-1)c} \leq v_{ic}$, for $1 < i \leq r$

The above two sets of criteria A and B sorts the data in a row-major snake-like pattern as illustrated in Fig. 1 (a). We also make the following assumptions:

- 1) The distance of separation between any two consecutive nodes is the same, r .
- 2) All the nodes have uniform communication radius, r .
- 3) A node s can communicate only with the nodes that are located within a distance r from it, i.e., it cannot communicate with another node s' (Fig. 1 (b)) that is diagonally located with respect to its own position.

B. Algorithm

The algorithm is iterative. Each iteration consists of four steps and in each step, a number of nodes group together and perform localized sorting among themselves. The possible number of nodes in a group can either be one, two or four. In the following, we describe the four steps illustrating the scenario with 16 nodes arranged in a 4×4 grid.

1) *Step 1:* In the first step, groups of four nodes are formed as shown in Fig. 2 (a). The top-left nodes (darkened in the figure) in each group act as leaders, in the sense that all other nodes within a group transmit their data to this node. The arrows shown in the figure signify local broadcasts within a group. For each group, it takes five local broadcasts to complete the following three steps:

- all the member nodes' data reach the leader
- the leader sorts the data locally
- the leader transmits the sorted data back to the member nodes.

These five local broadcasts will take 4 time slots because two of the broadcasts can happen parallelly (arrows marked as 1 in the figure). Time required for the leader to sort the four values is 6 time slots (it takes 6 comparisons in the worst case to sort 4 values). Thus, the total time taken for all the three steps to happen is 10 time slots. Note that, the local broadcasts in all the groups will take place in parallel. Hence, total time taken for step 1 to complete is 10 time slots.

2) *Step 2:* In this step, the groups will shift one column towards right from step 1, as illustrated in Fig. 2 (b) and the same process, as described in step 1, will continue. Here, groups of two nodes are formed in the first and last columns of the grid, when c is even and $c > 2$. On the other hand, when c is odd and $c > 1$, groups of two nodes are formed only in the first column. Time slots required to complete this step is also 10, because it essentially follows the same three steps as described in step 1.

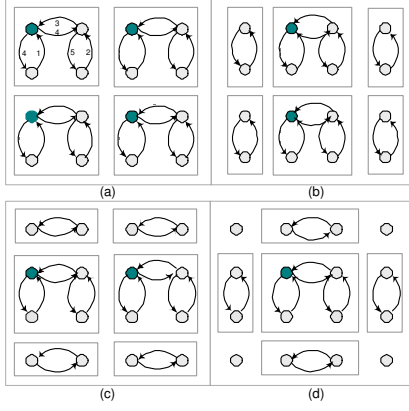


Fig. 2. (a) Step 1, (b) Step 2, (c) Step 3, (d) Step 4

3) *Step 3*: In the third step, the groups will shift one row towards down from step 1, as shown in Fig. 2 (c) and then the same process follows. Groups of two nodes are formed in the first and last rows, when r is even and $r > 2$, where as groups of two nodes are formed only in the first row when r is odd and $r > 1$. Time slots required to complete this step is again 10.

4) *Step 4*: This step is similar to step 2, where the groups are shifted one column towards right from step 3 and the same process of localized broadcast and sorting continues. Time required to complete this step is 10 slots.

The leader node preserves the previous iteration data of all the members in its group. Each of the members preserves its one-iteration data during its association with four different groups. The members will transmit the data to its leader only when there is a change from the previous value, except for the first time. The leader will understand that a node's value has not changed after the last iteration if the node does not transmit, and the leader proceeds with comparison with the previous data. If there is no change in the values for all the members they, as well as the leader do not transmit as the group is already sorted. But, if there is a change in leader's own data, it compares its data with the previous data of its three member nodes and broadcast the sorted data. The network is sorted, if all the groups are settled without any transmission and reception.

C. Complexity Analysis

Considering r rows and c columns we provide the following analysis of the algorithm. We use the following notations: t_{tr} is the time required for one transmission, t_c is the time required for one comparison, E_{tr} is the energy spent during one transmission and E_{rcv} is the energy spent during one reception.

1) *Step 1*: It will have $(\lfloor \frac{r}{2} \rfloor \lfloor \frac{c}{2} \rfloor)$ groups of 2×2 nodes formed, and $(r - 2\lfloor \frac{r}{2} \rfloor) + (c - 2\lfloor \frac{c}{2} \rfloor)$ groups of 2 nodes formed. Since sorting in this step requires 4 time slots for 5 local broadcasts and 6 time slots by the leader node for 6 comparisons, time complexity of step 1 will be $4t_{tr} +$

$6t_c$. The energy used will be $(\lfloor \frac{r}{2} \rfloor \lfloor \frac{c}{2} \rfloor) [5E_{tr} + 6E_{rcv}] + (r - 2\lfloor \frac{r}{2} \rfloor) [E_{tr} + E_{rcv}] + (c - 2\lfloor \frac{c}{2} \rfloor) [E_{tr} + E_{rcv}]$.

2) *Step 2*: It will have $(\lfloor \frac{r-1}{2} \rfloor \lfloor \frac{c}{2} \rfloor)$ groups of 2×2 nodes formed, and $(r - 2\lfloor \frac{r-1}{2} \rfloor) + (c - 2\lfloor \frac{c}{2} \rfloor)$ groups of 2 nodes formed. Since sorting in this step will also require 5 transmissions and 6 comparisons, time complexity of step 2 will be $4t_{tr} + 6t_c$. The energy used will be $(\lfloor \frac{r-1}{2} \rfloor \lfloor \frac{c}{2} \rfloor) [5E_{tr} + 6E_{rcv}] + (r - 2\lfloor \frac{r-1}{2} \rfloor) [E_{tr} + E_{rcv}] + (c - 2\lfloor \frac{c}{2} \rfloor) [E_{tr} + E_{rcv}]$.

3) *Step 3*: It will have $(\lfloor \frac{r}{2} \rfloor \lfloor \frac{c-1}{2} \rfloor)$ groups of 2×2 nodes formed, and $(r - 2\lfloor \frac{r}{2} \rfloor) + (c - 2\lfloor \frac{c-1}{2} \rfloor)$ groups of 2 nodes formed. Since sorting in this step requires 5 transmissions and 6 comparisons, time complexity of this step will be $4t_{tr} + 6t_c$. The energy used will be $(\lfloor \frac{r}{2} \rfloor \lfloor \frac{c-1}{2} \rfloor) [5E_{tr} + 6E_{rcv}] + (r - 2\lfloor \frac{r}{2} \rfloor) [E_{tr} + E_{rcv}] + (c - 2\lfloor \frac{c-1}{2} \rfloor) [E_{tr} + E_{rcv}]$.

4) *Step 4*: It will have $(\lfloor \frac{r}{4} \rfloor \lfloor \frac{c}{4} \rfloor)$ groups of 2×2 nodes formed, and $4(\lfloor \frac{r}{4} \rfloor \lfloor \frac{c}{4} \rfloor) + (r - 4\lfloor \frac{r}{4} \rfloor) + (c - 4\lfloor \frac{c}{4} \rfloor)$ groups of 2 nodes formed. Since sorting in this step requires 5 transmissions and 6 comparisons, time complexity of step 4 will be $4t_{tr} + 6t_c$. The energy used will be $(\lfloor \frac{r}{4} \rfloor \lfloor \frac{c}{4} \rfloor) [5E_{tr} + 6E_{rcv}] + (4(\lfloor \frac{r}{4} \rfloor \lfloor \frac{c}{4} \rfloor) + (r - 4\lfloor \frac{r}{4} \rfloor) + (c - 4\lfloor \frac{c}{4} \rfloor)) [E_{tr} + E_{rcv}]$.

D. Total time and energy complexity

Summing up the total number of transmissions and comparisons, the total time for each iteration will be $\mathcal{T} = (16t_{tr} + 24t_c)$, and the total energy spent will be $\mathcal{E} = \mathcal{O}(\lfloor \frac{r}{2} \rfloor \lfloor \frac{c}{2} \rfloor)$, as we can observe that $(\lfloor \frac{r}{2} \rfloor \lfloor \frac{c}{2} \rfloor)$ is the largest term in total energy calculation. Now, consider Fig. 3 (b), where we show the movement of a data element (darkened circle) from the right-bottom most position. If the data elements on the left and above this are all smaller, then over the first iteration this data will not move. However, from the second iteration and on each iteration thereafter, it will move atleast by 2 columns left and 2 rows upwards. This way the worst number of iterations will be $1 + \sqrt{(\lfloor \frac{r}{2} \rfloor)^2 + (\lfloor \frac{c}{2} \rfloor)^2}$. Thus, total time required for execution will be $\mathcal{T} = \left(1 + \sqrt{(\lfloor \frac{r}{2} \rfloor)^2 + (\lfloor \frac{c}{2} \rfloor)^2}\right) (16t_{tr} + 24t_c)$. For the case of square grid $r = c = \sqrt{n}$, we will have $\mathcal{T} = \left(1 + \sqrt{2} \lfloor \frac{\sqrt{n}}{2} \rfloor\right) (16t_{tr} + 24t_c) \cong \mathcal{O}(\sqrt{n})$, while energy will be $\mathcal{E} = \mathcal{O}(\lfloor \frac{r}{2} \rfloor \lfloor \frac{c}{2} \rfloor) \left(1 + \sqrt{(\lfloor \frac{r}{2} \rfloor)^2 + (\lfloor \frac{c}{2} \rfloor)^2}\right) = \mathcal{O}(n\sqrt{n})$.

III. GENERIC: RANDOM DEPLOYMENT, SINGLE-CHANNEL WSN

A. Framework

In this section, we formulate the ranking and sorting problems under a generic setup of distributed sensor networks. We consider a network of n sensor nodes distributed randomly on a two-dimensional plane and assume that each node has a unique identifier, each of which could be taken from an index set I . Thus, if S is the set of n sensor nodes, then each node is indexed by an element of the index set, and the set of sensor

identifiers is indicated by $S_I = \{s_i \mid \forall i \in I\}$. An ordering of these nodes is thus a permutation on this set indicated as:

$$S_{\pi(I)} = \{s_{\pi(i)} \mid \forall i \in I, i = 1, \dots, n\}$$

We assume that all the nodes transmit at the same power P , using an omnidirectional antenna and has a uniform communication radius r , satisfying $P_r = P/r^\alpha$, where $2 \leq \alpha \leq 5$ depending on environmental parameters and P_r is the received power at a distance r from the node. For a pair of nodes $u, v \in S$, if the Euclidian distance between them $\|u - v\| \leq r$, then there exists a bi-directional link between u and v . Thus, we model the sensor network as an undirected geometric random graph, $G = (V, E)$, where the vertex set V is the set of sensor nodes S and the edge set E is generated by the communication radius, such that if $u, v \in S, u \neq v$ then

$$e_{uv} = \begin{cases} 1, & \|u - v\| \leq r \\ 0, & \text{otherwise,} \end{cases}$$

and $e_{uv} \in E$ if and only if $e_{uv} = 1$. We further assume that by a proper selection of P , the network generated is one-connected, i.e., between any pair of nodes there exists a single hop or a multihop communication path. We provide the following definition for a node's neighborhood.

DEFINITION 3.1: Neighborhood: For a node u , we define its one-hop neighborhood as the set of nodes S_u^1 , with which u is able to communicate directly in a single hop, and its two-hop neighborhood as the set of nodes S_u^2 , with which u is able to communicate either in a single hop or in two hops. Thus,

$$S_u^1 = \{v \mid e_{uv} \in E, u \neq v\}; S_u^2 = \{S_u^1 \cup_{v \in S_u^1} S_v^1\}$$

Each node has knowledge only about its one-hop neighborhood, no other global picture is assumed.

Every sensor node observes and measures certain physical quantity of interest, which could be temperature, pressure, humidity, chemical contamination or acoustic energy dissipated by a moving vehicle. Let us denote the measurement of a node u as m_u . A ranking problem for such a network can now be defined as a set ordering π on S_I , which generates $S_{\pi(I)}$ such that if two nodes $u, v \in S$ have measurements m_u and m_v respectively, and $m_u \geq m_v$, then $s_{\pi(u)} \geq s_{\pi(v)}$, where $s_{\pi(u)}, s_{\pi(v)} \in S_I$, and we call this a non-decreasing order. Similarly, a non-increasing ordering can be defined as an ordering that ensures $s_{\pi(u)} \leq s_{\pi(v)}$, when $m_u \leq m_v$. We assume that ranking is performed on a snapshot of the network and measurement does not change during the ranking process. Note that, this global ordering is generated on the identifier set S_I , and hence, hold the following properties.

- 1) Only an ordering of the identifiers is generated and thus one can query the network with identifier and get the sorted values.
- 2) The measurement of a node has not changed due to ranking, only an ordering is created.

Next, we define the notions of connectivity matrix, interference graph, feasibility matrix and feasible transmission set for

framing the sorting problem in sensor networks and proving it to be NP-hard.

DEFINITION 3.2: Connectivity Matrix (C): We represent the sensor network graph $G = (V, E)$ in the form of a $n \times n$ symmetric matrix, called the Connectivity Matrix $C = (c_{ij})$; $i, j = 1, \dots, n$, where $n = |V|$ and

$$c_{ij} = \begin{cases} 1, & e_{ij} \in E \text{ \& } i \neq j \\ 0, & \text{otherwise.} \end{cases}$$

The notion of interference graph comes from the hidden terminal problem that occurs because of the broadcast nature of communication in sensor networks. A node with a single antenna can receive data from only one of its neighbors at a particular time instant. Hence, hidden terminal problem arises when two nodes, which are not in each other's neighborhood, transmit data to the same node, which is a common neighbor to both of them, at the same time instant. Thus, when a node u is transmitting to one of its neighbors v , no other node within the two hop neighborhood (S_u^2) of u should transmit to v at the same time.

DEFINITION 3.3: Interference Graph (G_I): The Interference Graph is defined as the augmented graph $G_I = (V_I, E_I)$ on $G = (V, E)$, such that the set of vertices V_I consists of all the sensor nodes and set of edges E_I consists of all the edges in the original graph G as well as, the new edges between every node and its two hop neighbors. Thus,

$$V_I = V; \text{ \& } E_I = E \cup \bigcup_{\forall (u \in V_I, v \in S_u^2, v \notin S_u^1)} e_{uv}.$$

DEFINITION 3.4: Simultaneous Transmission Feasibility Matrix (F): We represent a "complimentary" $n \times n$ symmetric matrix $F = (f_{ij})$; $i, j = 1, \dots, n$, with respect to the interference graph G_I , such that $n = |V_I|$ and

$$f_{ij} = \begin{cases} 1, & e_{ij} \notin E_I \text{ \& } i \neq j \\ 0, & \text{otherwise.} \end{cases}$$

Note that, $f_{ij} = f_{ji}$ for this matrix and indicates that if $f_{ij} = f_{ji} = 1$, then both i and j can transmit at the same time instant without collision. Each of the column vectors of F is called a feasible transmission set, denoted by $\tau = (t_i)$; $i = 1, \dots, n$, and $t_i = 1$ signifies that node i can transmit without causing collision.

Each step of execution of a ranking or sorting algorithm will consist of several such feasible transmission sets. Thus, we can express the execution of a ranking or sorting algorithm with a $n \times p$ matrix T , each column of which represents a feasible transmission set τ_j , $j = 1, \dots, p$. We make the following observations.

- 1) The value of p is not necessarily polynomially bounded in the number of nodes, n . The optimal ranking or sorting algorithm will have the minimum value of p .
- 2) For a node i to participate in the ranking or sorting algorithm the condition $\sum_{j=1}^p t_{ij} \geq 1$, must hold.

B. Complexity of the sorting or ranking algorithm

In this section, we analyze the complexity of the ranking or sorting algorithm that has been formulated in the previous section as an ordering of p feasible transmission sets.

1) *Problem Instance: SORT-WSN*: Given are the following:

- A WSN $G = (V, E)$,
- A function $m : V \rightarrow \mathbb{Z}^+$,
- An ordering function $\pi : V \rightarrow V$, such that $\forall i, j \in V$, $\pi(i) < \pi(j) \Rightarrow m(i) \leq m(j)$, the ranks are ordered, i.e., $m_i \leq m_j$,
- A positive integer k ,
- A $N \times M$ matrix T , where each column is a column vector $\vec{t}_i$ - a feasible transmission set.

2) *Question*: Is there an arrangement of the feasible transmission set vectors $\vec{t}_i$ to construct T such that $i \leq k$ and $\pi(v)$ is achieved?

Theorem 3.5: Sorting in WSN under the generic framework described in section III-A is NP-hard.

Proof: It is easy to see that the sorting problem is in NP, because given a $N \times M$ schedule it is easy to verify in polynomial time whether this constructs a collision-free schedule whose length is smaller than k , and whether this leads to a sorted arrangements of the values in the network.

Next, it is known that the Graph Colorability problem (GC) is in NP and also it is NP-Complete. Thus, it follows that the colorability problem of the interference graph as defined in section 3.3, can be Turing reducible to formation of one round feasible schedule, such that in that round every node gets a chance to transmit, i.e.,

However, M is not polynomially bounded in the number of inputs N for the sorting problem and hence, for any problem $\mathcal{A}$ in NP, SORT-WSN $\mathcal{A}$. That is, a polynomial time reduction (Karp Reduction) cannot be shown to any problem in NP from SORT-WSN. Hence, SORT-WSN is NP-hard. ■

C. A Randomized Algorithm for SORT-WSN

Having proved that the problem SORT-WSN is NP-hard, we now describe a randomized sorting algorithm that proves the existence of a schedule of exponential length. We consider $G_I(n, p)$ as an interference graph of a WSN graph G with n nodes, where each interference edge in G_I has been included between every pair of nodes in G with probability p ; i.e., $\forall u, v \in G$, $\Pr[(u, v) \in E_I] = p$.

We define each feasible transmission set be generated over $[0, 1]^n$, as a string R . Imagine that someone tosses a fair coin n times to generate a feasible set and include node i in the set to transmit iff the outcome of the i^{th} coin toss is 1, i.e., $R_i = 1$.

Theorem 3.6: For every $G_I(n, p)$, there exists a schedule that allows almost all nodes to transmit once, and is e^{np} long.

Proof: We assign a random variable X_i with each i^{th} node's inclusion to a feasible transmission set, such that:

$$X_i = \begin{cases} 1, & \text{node } i \text{ can transmit without collision \& } R_i = 1 \\ 0, & \text{otherwise.} \end{cases}$$

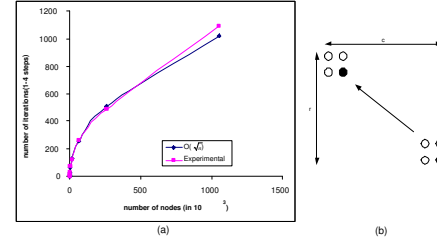


Fig. 3. (a) Simulation results for $\sqrt{n} \times \sqrt{n}$ lattice networks for $n = 2^2$ to $n = 1024^2$ (b) Worst case movement.

and $X = \sum_{i=1}^n X_i$, indicating how many node feasibly get scheduled in one such time interval. Clearly $E(X) = E(\sum_{i=1}^n X_i) = \sum_{i=1}^n E(X_i)$. Now $E(X_i) = \Pr[X_i = 1] = \prod_{j \in n, j \neq i} \Pr[(i, j) \in E_I \cap R_j = 1] = (p/4)^{n-1}$. Thus, $E(X) = n(p/4)^{n-1} \geq ne^{-n(1-p/4)}$. Now, if we perform this process of random feasible schedule creation for m times and assign a random variable $Y = \sum_{i=1}^m X_i$, then $E(Y) = \sum_{i=1}^m E(X_i) = mne^{-n(1-p/4)}$. Since $\Pr[X > E(Y)] > 0$, we claim that this process will create a schedule where atleast $mne^{-n(1-p/4)}$ nodes will get a chance to transmit atleast once. ■

IV. SIMULATION AND RESULTS

We used MATLAB to simulate the sorting algorithm presented in II. In our simulation, we considered square grids of sizes 2×2 , 4×4 and so on upto 1024×1024 , which means we simulated a network with maximum 2^{20} nodes. In every run with more than 16×16 nodes, we conducted the experiments 1000 times and plotted the number of iterations in the worst case, required for the network to get sorted. We also plotted the graph for $\sqrt{n}$ and found out that the worst case time complexity is close to $\sqrt{n}$, neglecting the constant factors. The graphs are shown in Fig. 3 (a).

V. CONCLUSIONS

In this paper, we proposed a simple, parallel and localized sorting algorithm for WSN that runs in time complexity $O(\sqrt{n})$. We also formally presented the sorting/ranking problem under the generic setup of single-channel random deployment of nodes and proved that finding an optimal schedule is NP-hard. Lastly, we proved the existence of a randomized sorting algorithm under the generic setup. In our future work, we plan to extend the localized algorithm for the generic setup and to provide theoretic bounds of sorting/ranking problem in WSN.

REFERENCES

- [1] J. L. Bordim, K. Nakano, and H. Shen, "Sorting on single-channel wireless sensor networks," *International Journal on Foundations of Computer Science*, vol. 14, no. 3, pp. 391–403, June 2003.
- [2] D. Nassimi and S. Sahni, "Bitonic sort on a mesh-connected parallel computer," *IEEE Transactions on Computers*, vol. 28, no. 1, pp. 2–7, Jan. 1979.
- [3] M. Singh, V. Prasanna, J. Rolim, and C. Raghavendra, "Collaborative and distributed computation in mesh-like wireless sensor arrays," in *Personal Wireless Communications (PWC'03)*, LNCS 2775, Venice, Italy, Sept. 2003, pp. 1–11.